

Universidad de Costa Rica
Facultad de Ingeniería
Escuela de Ingeniería Eléctrica
IE0435 - Inteligencia Artificial Aplicada a la Ingeniería Eléctrica
Grupo 01
I ciclo 2026

Reporte para el proyecto 2

Estudiante:
Caleb Arias Fallas C20739

Profesor: Marvin Coto Jiménez

Índice

1. Introducción	2
2. Herramientas originales del agente	2
3. Integración de las nuevas herramientas	3
3.1. Herramientas para estadísticas completas <code>get_statistics</code>	3
3.2. Herramienta para calcular estadísticas horarias históricas.	4
3.3. Herramienta de visualización estadística	4
4. Cambios realizados en <code>main2.py</code>	4
5. Limitaciones y recomendaciones	5
6. Resultados obtenidos	5
6.1. Estadísticas completas de la columna MW	5
6.2. Promedio histórico por hora de la columna MW	6
6.3. Histograma de la columna MW	7
6.4. Boxplot de la columna MW	7
6.5. Gráfico del promedio histórico por hora	7
7. Conclusiones y Recomendaciones	8

1. Introducción

El proyecto consiste en la implementación de herramientas para un agente que se encarga del análisis de datos de consumo eléctrico del país por medio de la información proporcionada por el ICE. El agente trabaja sobre la demanda eléctrica con una frecuencia de muestreo donde se obtiene un valor de consumo cada quince minutos.

El proyecto pretende darle al agente nuevas herramientas, para el caso específico se cubren las herramientas para obtener valores estadísticos de los datos en general para calcular distintas métricas así como una herramienta para obtener información estadística utilizando los históricos horarios, así como una tercera herramienta de visualización para estadísticas.

Adicionalmente se realiza una tercera herramienta para poder graficar resultados obtenidos por las dos anteriores y evitar conflictos con herramientas actuales que se encuentran implementadas en el agente base.

2. Herramientas originales del agente

El agente base trae consigo las herramientas listadas a continuación:

- `code_exec`
- `plot_data`
- `predict_data`
- `final_answer`

Estas herramientas están estructuradas en un diccionario llamado `dict_tools` en el módulo `main2.py` que contiene las funciones principales del agente. Carga los datos de consumo, configura el modelo de lenguaje local utilizado que en este caso corresponde a `ollama`, define las herramientas base disponibles y maneja la interacción con el usuario. La primera herramienta listada llamada `code_exec` está diseñada para ejecutar instrucciones de código sobre el conjunto de datos cargados en memoria cuando se corre el programa. Esta herramienta permite realizar cálculos sobre el dataframe definido y es capaz de manejar algunas métricas básicas como máximos, mínimos o promedios.

La herramienta `plot_data` se encarga de generar gráficos a partir de las columnas del `DataFrame`. Esta función permite seleccionar columnas específicas, filtrar por fechas de inicio y fin, y construir un gráfico temporal con etiquetas para el eje de tiempo y potencia eléctrica. Esta herramienta permite visualizar el comportamiento y la evolución del consumo eléctrico. Además, se incluye una función auxiliar llamada `normalize_plot_args`, que corrige el formato de los argumentos cuando el modelo entrega las columnas como texto en lugar de lista.

La herramienta `predict_data` permite generar predicciones de series temporales utilizando los modelos `Prophet` y `SARIMA`. Para ello, la función recibe como parámetros el modelo a utilizar, la columna que se desea predecir, el horizonte de predicción y, opcionalmente, una fecha final. En el caso de `Prophet`, se ajusta un modelo con estacionalidad diaria y se generan valores futuros con frecuencia de quince minutos. En el caso de `SARIMA`, se utiliza una configuración

con componentes estacionales asociados a los 96 intervalos de quince minutos que conforman un día, finalmente esta herramienta también puede realizar gráficos de los valores futuros.

Finalmente la herramienta `final_answer` que tiene como labor devolver una respuesta en lenguaje natural al usuario. Esta herramienta se utiliza cuando el agente ya tiene la información necesaria para responder o cuando la consulta no requiere cálculos adicionales. También se define su estructura como herramienta compatible con el sistema de llamadas de funciones, indicando el nombre, la descripción y los parámetros esperados.

3. Integración de las nuevas herramientas

El módulo `new_features.py` es un módulo que encapsula las herramientas añadidas en el proyecto, se optó realizar un nuevo módulo para mantener los cambios del archivo original al mínimo. En este archivo se incorporan las herramientas orientadas al análisis estadístico, la generación de perfiles históricos y visualización de resultados estadísticos. De esta manera, el módulo complementa al agente original, permitiéndole responder consultas más específicas sobre el comportamiento de los datos eléctricos almacenados en el DataFrame global `dtf`.

El módulo incorpora una función llamada `show_plots_and_wait`, la cual revisa si existen figuras abiertas y, en caso afirmativo, las muestra bloqueando la ejecución hasta que el usuario cierre las ventanas. Esta función es importante porque permite que el agente espere a que el usuario termine de observar los gráficos antes de continuar con otra consulta. Esta función fue necesaria debido a que en pruebas preliminares habían momentos en los cuáles se abrían múltiples figuras o el `prompt` quedaba corriendo en paralelo yabría la misma figura múltiples veces.

3.1. Herramientas para estadísticas completas `get_statistics`

La primera herramienta principal definida en este módulo es `get_statistics`. Esta función se encarga de calcular estadísticas de la columna especificada del dataset. Antes de realizar los cálculos, valida que el DataFrame exista en memoria, ya que este módulo no carga directamente el archivo CSV, sino que depende de que el dataset ya haya sido cargado previamente por el módulo principal. También verifica que la columna solicitada exista dentro del DataFrame y que contenga datos válidos. Si el usuario no especifica una columna, la función intenta seleccionar automáticamente la primera columna numérica disponible.

La herramienta revisa la existencia de la columna y se definen las métricas que se pueden calcular como media, mediana, desviación estándar, varianza, valor mínimo, valor máximo, rango, primer cuartil, tercer cuartil, rango intercuartílico, asimetría, curtosis, cantidad total de datos, cantidad de datos faltantes y porcentaje de datos faltantes. Estos resultados permiten conocer de forma general cómo se comporta una variable como MW o MW_P, identificando su tendencia central, dispersión, distribución y calidad de los datos.

Además de las estadísticas generales, la herramienta `get_statistics` permite agrupar los resultados por hora o por día mediante los parámetros `by_hour` y `by_day`. Cuando selecciona el agrupamiento por hora, el módulo calcula estadísticas como media, desviación estándar, mínimo, máximo y conteo para cada hora del día. Cuando se activa el agrupamiento por día, se puede observar el comportamiento diario de la variable, permitiendo comparar la demanda

entre diferentes fechas.

3.2. Herramienta para calcular estadísticas horarias históricas.

La segunda herramienta importante es `historical_hourly`. Esta función calcula valores estadísticos por hora de manera histórica utilizando una columna especificada. A diferencia de `get_statistics`, que entrega un resumen estadístico más general, `historical_hourly` se enfoca en el comportamiento promedio de la variable durante las veinticuatro horas del día. Para ello, agrupa los datos según la hora del índice temporal y calcula la media, desviación estándar, mínimo y máximo para cada hora. Esto permite construir una tabla donde se resume cómo varía la demanda eléctrica promedio a lo largo del día.

3.3. Herramienta de visualización estadística

La tercera herramienta es `stat_plotting`, encargada de generar gráficos de métricas estadísticas a partir de una columna especificada del dataset. Esta función permite mostrar diferentes tipos de gráficos según el parámetro `plot_type` que se solicite en el prompt. Entre las opciones disponibles están gráficos de distribución, histogramas, boxplots.

El histograma permite observar la frecuencia con la que aparecen ciertos valores de demanda, mientras que el boxplot facilita identificar la dispersión, la mediana y posibles valores atípicos. Si se selecciona únicamente histograma o boxplot, la función genera solo el gráfico correspondiente, si se especifican ambos se hacen las dos figuras.

Lo último que incluye el módulo `new_features` son estructuras para la integración de las nuevas herramientas. La primera función es `new_tools`, un diccionario que asocia los nombres de las nuevas herramientas con sus respectivas funciones de Python. La segunda es `new_tools_schemas`, que permite que el modelo de lenguaje pueda identificar y llamar correctamente las herramientas. La tercera es `normalizers`, que relaciona cada herramienta con su función normalizadora correspondiente. Estas estructuras permiten que el módulo pueda ser incorporado al sistema principal sin mezclar directamente todas las funciones dentro de `main2.py`.

4. Cambios realizados en `main2.py`

Para integrar las herramientas del módulo `new_features.py`, el archivo `main2.py` tuvo que ser ajustado para incluir las herramientas nuevas que se encuentran en dicho módulo. Se agregó `new_features.dtf`, con el fin de compartir el DataFrame ya cargado con las nuevas herramientas. Esto evita volver a leer el archivo CSV y permite que tanto las funciones originales como las nuevas trabajen sobre el mismo conjunto de datos.

También se amplió el diccionario principal de herramientas con una llamada de actualización, lo que incorpora las tres herramientas estadísticas nuevas a la ejecución del agente. Además, en el bucle principal se agregan los esquemas definidos en `new_features.new_tools_schemas`, para que el modelo de lenguaje pueda reconocer y utilizar estas herramientas durante la interacción con el usuario.

Otro cambio importante fue la lógica de normalización y redirección de herramientas donde se agregaron funciones para detectar solicitudes de gráficos estadísticos, inferir sus argumentos y redirigirlas hacia la herramienta correcta; esto evita que el agente use por error una herramienta de cálculo cuando en realidad se solicita una visualización.

Finalmente, el prompt del sistema fue actualizado para incluir las nuevas herramientas y las reglas de uso. Por ejemplo, si el usuario pide calcular estadísticas, debe usar `get_statistics`, si solicita el promedio histórico por hora, debe usar `historical_hourly`, y si pide graficar histogramas, boxplots o perfiles horarios, debe usar `stat_plotting`. Con estos cambios, `main2.py` conserva las funciones originales en general, pero ahora también puede realizar análisis estadísticos y visualizaciones especializadas sin generar conflictos con las herramientas anteriores.

5. Limitaciones y recomendaciones

Existen muchas limitaciones para pulir el agente y que pueda realizar tareas de más propósito general, la primera es que en general con las nuevas herramientas un prompt ambiguo puede generar resultados no deseados en cuanto a la herramienta que se debe ejecutar.

La herramienta para visualización de datos estadísticos tiene plots limitados que se deben especificar en el prompt cuando se pida la visualización, en este caso un punto para la mejora del algoritmo sería realizar un wrapper de `matplotlib` y sus gráficas estándar para poder visualizar correctamente cualquier solicitud en cuanto a los datos, también es limitado el número de figuras y en general el manejo de las ventanas no está completamente optimizado.

Otra dependencia importante es que el agente utiliza el dataset cargado y no se conecta en tiempo real a la base de datos actualizada del ICE para obtener datos actualizados constantemente y ser capaz de realizar predicciones, visualización y cálculo de métricas sobre los datos actuales en tiempo real. Lo ideal sería que el agente tuviera acceso a los datos actualizados y poder realizar cálculos y predicciones en tiempo real así como aprovechar las métricas de la herramienta proporcionada para tomar decisiones y avisar de eventos poco comunes en comparación con la información disponible de periodos horarios anteriores.

6. Resultados obtenidos

En esta sección se resumen las pruebas realizadas con las nuevas herramientas del agente. Para cada caso se utilizó una solicitud en lenguaje natural para activar una de las herramientas incorporadas: `get_statistics`, `historical_hourly` o `stat_plotting`. Las pruebas buscaban comprobar que el agente interpretara correctamente la petición del usuario, seleccionara la herramienta adecuada y devolviera una respuesta esperada con el tipo de análisis solicitado. A continuación, cada prompt se asocia con la imagen correspondiente de la prueba ejecutada.

6.1. Estadísticas completas de la columna MW

El primer prompt utilizado fue:

“Dame las estadísticas completas de la columna MW”

Se espera que el agente utilice la herramienta `get_statistics` y devuelva una tabla o respuesta textual con las estadísticas principales de la columna MW, incluyendo media, mediana, desviación estándar, varianza, valor mínimo, valor máximo, rango, cuartiles, rango intercuartílico, asimetría, curtosis, cantidad total de datos, datos faltantes y porcentaje de datos faltantes.

```

> Dame las estadísticas completas de la columna MW
> code_exec -> Inputs: {'code': "get_statistics(dtf['MW'])"}
Resultado:
=====
ESTADÍSTICAS DE 'MW'
=====
Media Mediana Desviación Estándar Varianza Mínimo Máximo Rango Q1 (25%) Q3 (75%) IQR Asimetría Curtosis Total Datos Datos Faltantes
Porcentaje Faltantes
MW 1475.14 1519.45 255.49 65273.48 838.76 1940.23 1101.47 1229.04 1700.88 471.85 -0.29 -1.21 35136 0
0.0
=====
Estadísticas calculadas para 'MW'. Media: 1475.14, Mediana: 1519.45, Desv.Std: 255.49

```

Figura 1: Resultado obtenido al solicitar estadísticas completas de la columna MW.

6.2. Promedio histórico por hora de la columna MW

El segundo prompt utilizado fue:

“Calcula el promedio histórico por hora de la columna MW e identifica la hora pico y la hora de menor demanda.”

Se espera que el agente utilice la herramienta `historical_hourly` y devuelva una tabla con el promedio, la desviación estándar, el valor mínimo y el valor máximo de MW para cada hora del día, además de identificar la hora pico y la hora de menor demanda según el promedio histórico.

```

> Calcula el promedio histórico por hora de la columna MW e identifica la hora pico y la hora de menor demanda.
> historical_hourly -> Inputs: {'column': 'MW'}
PERFIL HORARIO HISTÓRICO
=====
Hora Media Desv_Std Mínimo Máximo
0 1166.02 61.07 908.39 1335.08
1 1117.79 57.38 872.51 1256.51
2 1089.50 55.30 856.56 1220.93
3 1088.78 55.49 851.99 1211.39
4 1141.87 74.15 878.64 1317.79
5 1238.69 129.73 866.76 1484.75
6 1334.37 161.55 838.76 1573.55
7 1423.28 133.24 888.28 1636.05
8 1522.59 116.03 961.86 1705.74
9 1602.35 108.36 1075.67 1784.21
10 1659.00 110.77 1177.35 1852.28
11 1733.06 115.48 1253.16 1932.69
12 1746.55 106.33 1313.86 1923.87
13 1733.02 116.64 1327.32 1940.23
14 1699.53 125.62 1297.61 1932.27
15 1673.81 127.13 1251.13 1906.16
16 1646.23 123.30 1235.43 1875.97
17 1632.27 104.32 1252.79 1822.67
18 1715.27 102.40 1358.58 1915.94
19 1694.44 101.03 1326.22 1915.57
20 1622.56 93.97 1257.17 1827.42
21 1507.20 91.99 1167.93 1736.11
22 1365.05 78.74 1062.89 1587.48
23 1250.18 68.88 978.00 1427.25
=====
Hora de mayor demanda: 12:00 (1746.55)
Hora de menor demanda: 3:00 (1088.78)
Resultado:
Perfil horario calculado para MW. Hora pico: 12:00
> Perfil horario calculado para MW. Hora pico: 12:00

```

Figura 2: Resultado obtenido al calcular el promedio histórico por hora de la columna MW.

6.3. Histograma de la columna MW

El tercer prompt utilizado fue:

“Genera únicamente un histograma de la columna MW”

Se espera que el agente utilice la herramienta `stat_plotting` con el tipo de gráfico `histogram` y genere únicamente un histograma de la columna MW, mostrando la frecuencia de los distintos rangos de demanda eléctrica.

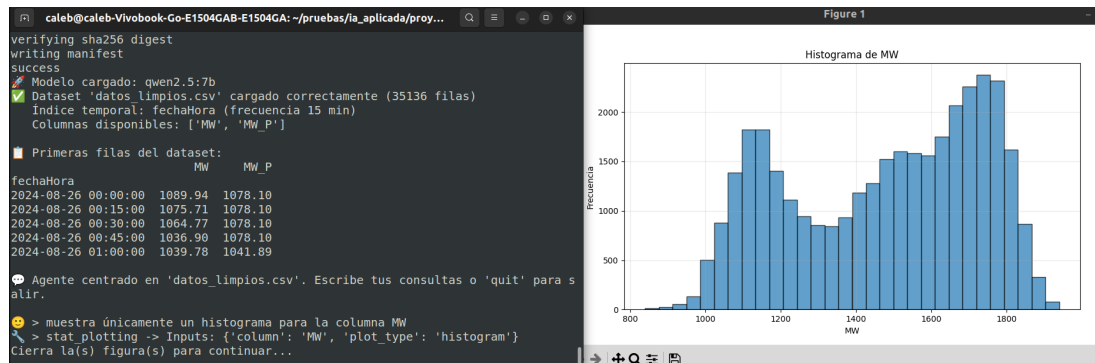


Figura 3: Histograma generado para la columna MW.

6.4. Boxplot de la columna MW

El cuarto prompt utilizado fue:

“Genera únicamente un boxplot de la columna MW”

Se espera que el agente utilice la herramienta `stat_plotting` con el tipo de gráfico `boxplot` y genere únicamente un diagrama de caja para la columna MW, mostrando la mediana, los cuartiles, la dispersión de los datos y los posibles valores atípicos.

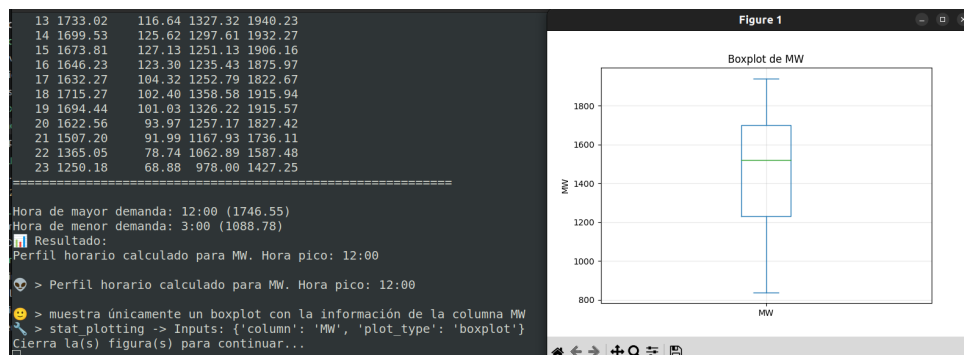


Figura 4: Boxplot generado para la columna MW.

6.5. Gráfico del promedio histórico por hora

El quinto prompt utilizado fue:

“Grafica el promedio histórico por hora de la columna MW.”

Se espera que el agente utilice la herramienta `stat_plotting` con el tipo de gráfico `hourly_profile` y genere una curva del promedio histórico de MW para cada hora del día, de forma que se visualicen los periodos de mayor y menor consumo promedio.

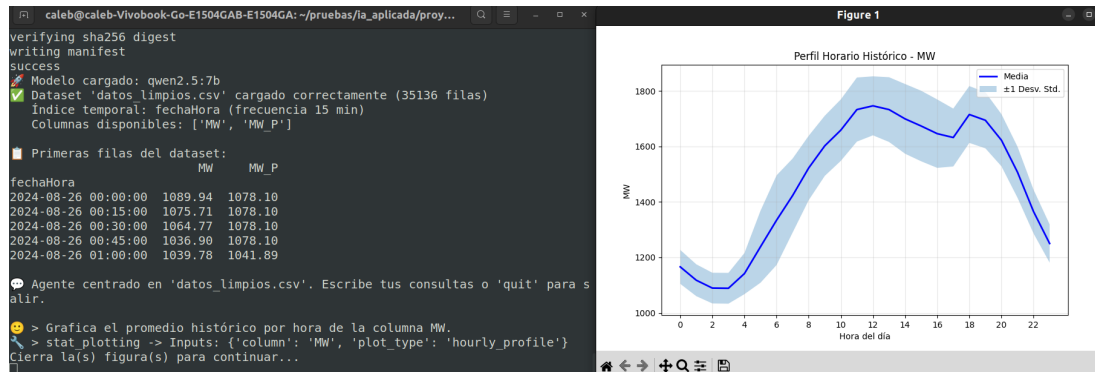


Figura 5: Gráfico del promedio histórico por hora de la columna MW.

En conjunto, estas pruebas muestran que las herramientas añadidas cumplen tres funciones: `get_statistics` entrega estadísticas descriptivas más completas, `historical_hourly` calcula el comportamiento horario histórico y `stat_plotting` genera visualizaciones específicas como histogramas, boxplots y perfiles horarios. La separación entre cálculo y visualización mejora el control del flujo del agente y reduce ambigüedades al interpretar los prompts del usuario.

7. Conclusiones y Recomendaciones

Se logró ampliar las herramientas del agente para que sea capaz de hacer un análisis estadístico con mayor alcance así como proporcionar una mejor visualización de datos y de comportamiento de consumo eléctrico. Además se realizó una integración que debería permitir el uso normal de las herramientas con las que el agente ya contaba como realizar predicciones utilizando cualquiera de los dos modelos disponibles para esto, la separación de las herramientas en un módulo separado permite realizar mantenimiento de manera más sencilla ya que no requiere cambiar el módulo principal.

A pesar de que el proyecto cumple con la implementación de las herramientas tiene múltiples puntos de mejora, por ejemplo separar las herramientas de código en módulos para dejar la implementación únicamente del agente en un módulo que incluya todos los demás módulos de herramientas para que el mantenimiento del proyecto en general sea menos complicada y escalable además. También se debe realizar un wrapper de la librería `matplotlib` para realizar una herramienta de visualización completa de propósito general, añadir gráficas con mejores especificaciones para graficar cualquier set de columnas entre sí.

Otro punto de mejora es que se pueda realizar análisis de datos en tiempo real conectándose directamente a la plataforma que posee el ICE para redirigir las actualizaciones en tiempo real hacia el modelo y poder realizar predicciones en tiempo real así como análisis estadístico, sin embargo esto requiere un modelo más grande y que consuma más recursos, además mayor capacidad de almacenamiento para tener disponible un record histórico.